Tim Pengajar IF2150

IF2150 – Rekayasa Perangkat Lunak

Component Level Design

SEMESTER I TAHUN AJARAN 2025/2026

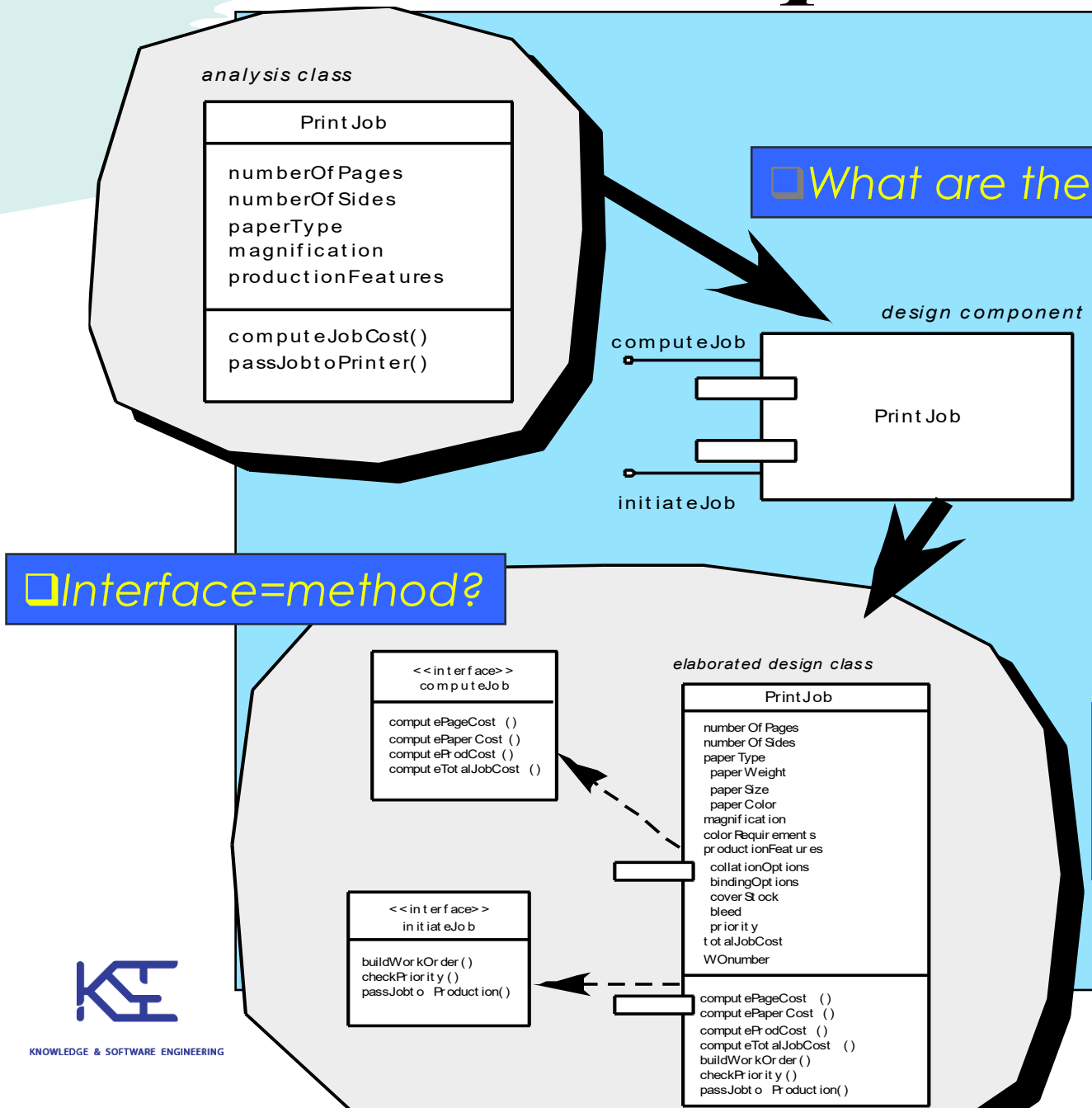
KNOWLEDGE & SOFTWARE ENGINEERING

What is a Component?

- *OMG Unified Modeling Language Specification* [OMG01] defines a **component** as
  - "… a modular, deployable, and replaceable part of a system that encapsulates *implementation* and exposes a set of *interfaces*."
- **OO view**: a component contains ***a set of collaborating classes***
- **Conventional view**: *logic*, the internal ***data structures*** that are required to implement the processing logic, and an ***interface*** that enables the component to be invoked and data to be passed to it.

What would be an example?

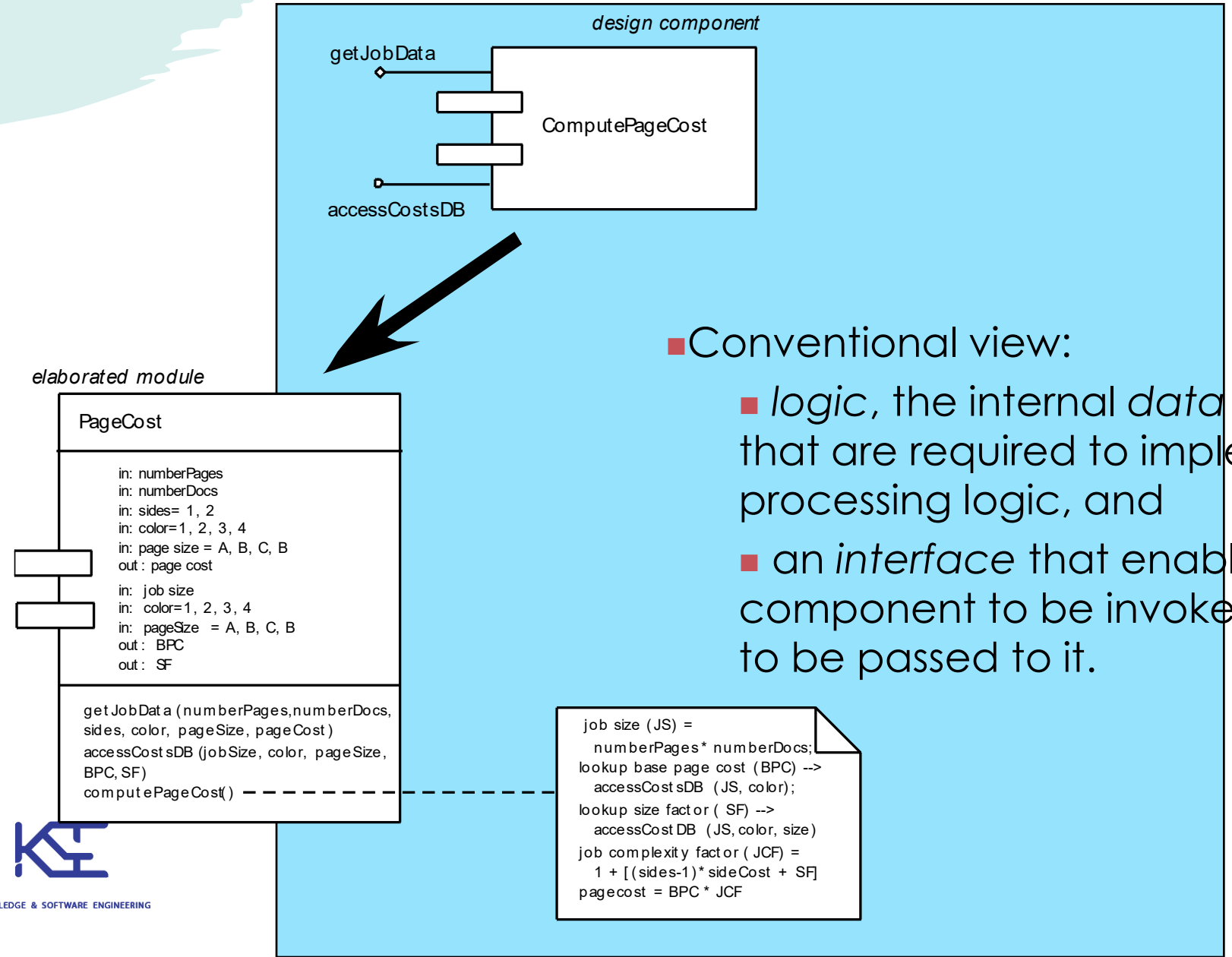
OO Component



OO view: a component contains a set of *collaborating classes*

KNOWLEDGE & SOFTWARE ENGINEERING

Conventional Component



- Conventional view:
  - *logic*, the internal *data structures* that are required to implement the processing logic, and
  - an *interface* that enables the component to be invoked and data to be passed to it.

KNOWLEDGE & SOFTWARE ENGINEERING

Cohesion

- Conventional view:
  - the "single-mindedness" of a module
- OO view:
  - a component or class encapsulates only attributes and operations that are closely related to one another

Coupling

- Conventional view:
  - The degree to which a *component* is connected to other components and to the external world
- OO view:
  - a qualitative measure of the degree to which *classes* are connected to one another

Designing class based component

Basic Design Principles

- The Open-Closed Principle (OCP). *"A module [component] should be open for extension but closed for modification.*
- The Liskov Substitution Principle (LSP). *"Subclasses should be substitutable for their base classes.*
- Dependency Inversion Principle (DIP). *"Depend on abstractions. Do not depend on concretions."*
- The Interface Segregation Principle (ISP). *"Many client-specific interfaces are better than one general purpose interface.*
- The Release Reuse Equivalency Principle (REP). *"The granule of reuse is the granule of release."*
- The Common Closure Principle (CCP). *"Classes that change together belong together."*
- The Common Reuse Principle (CRP). *"Classes that aren't reused together should not be grouped together."*

Source: Martin, R., "Design Principles and Design Patterns," downloaded from http:www.objectmentor.com, 2000.

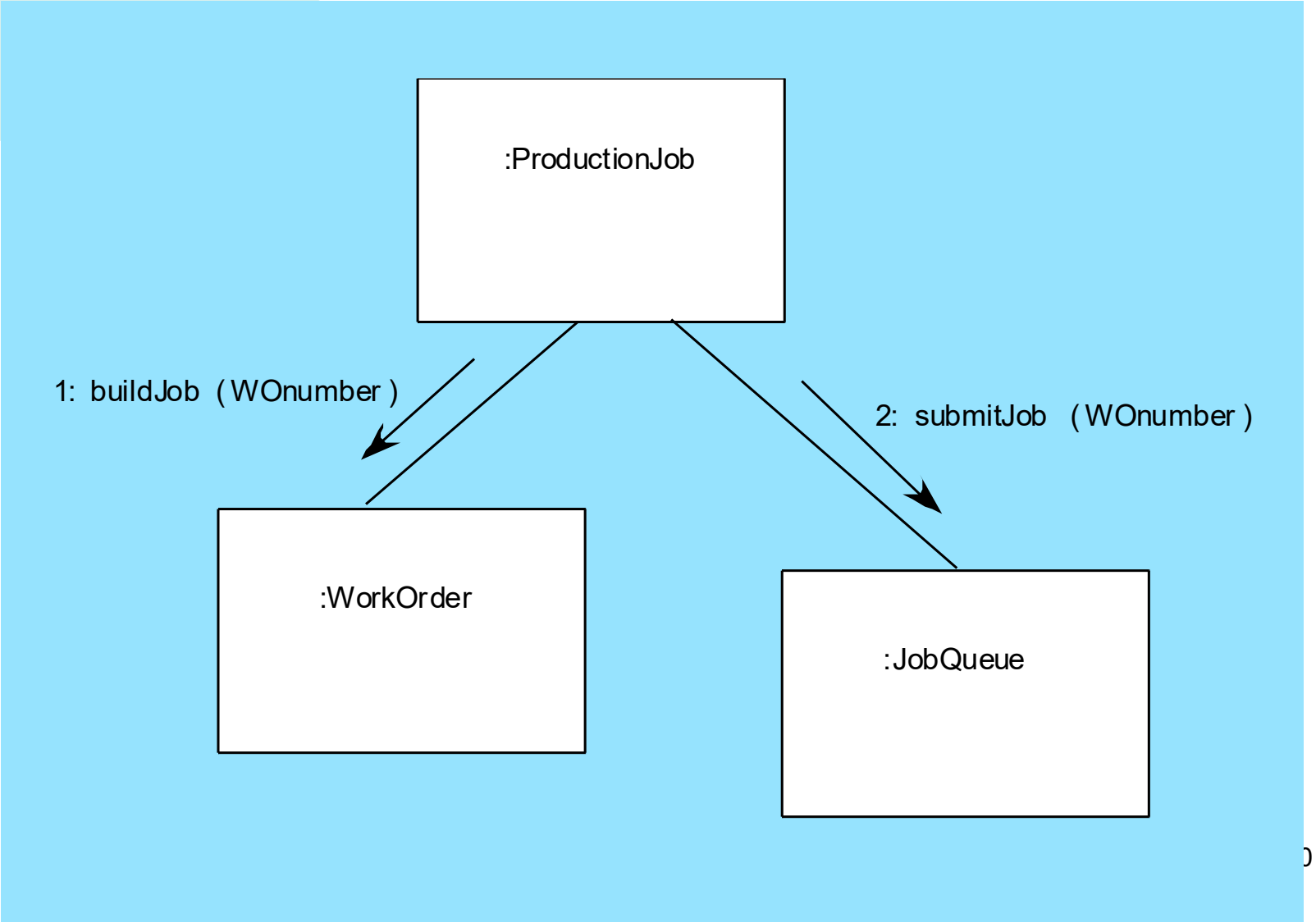
Design Guidelines

- **Components**
  - Naming conventions should be established for components that are specified as part of the architectural model and then refined and elaborated as part of the component-level model
- **Interfaces**
  - Interfaces provide important information about communication and collaboration
- **Dependencies and Inheritance**
  - it is a good idea to model dependencies from left to right and inheritance from bottom (derived classes) to top (base classes).

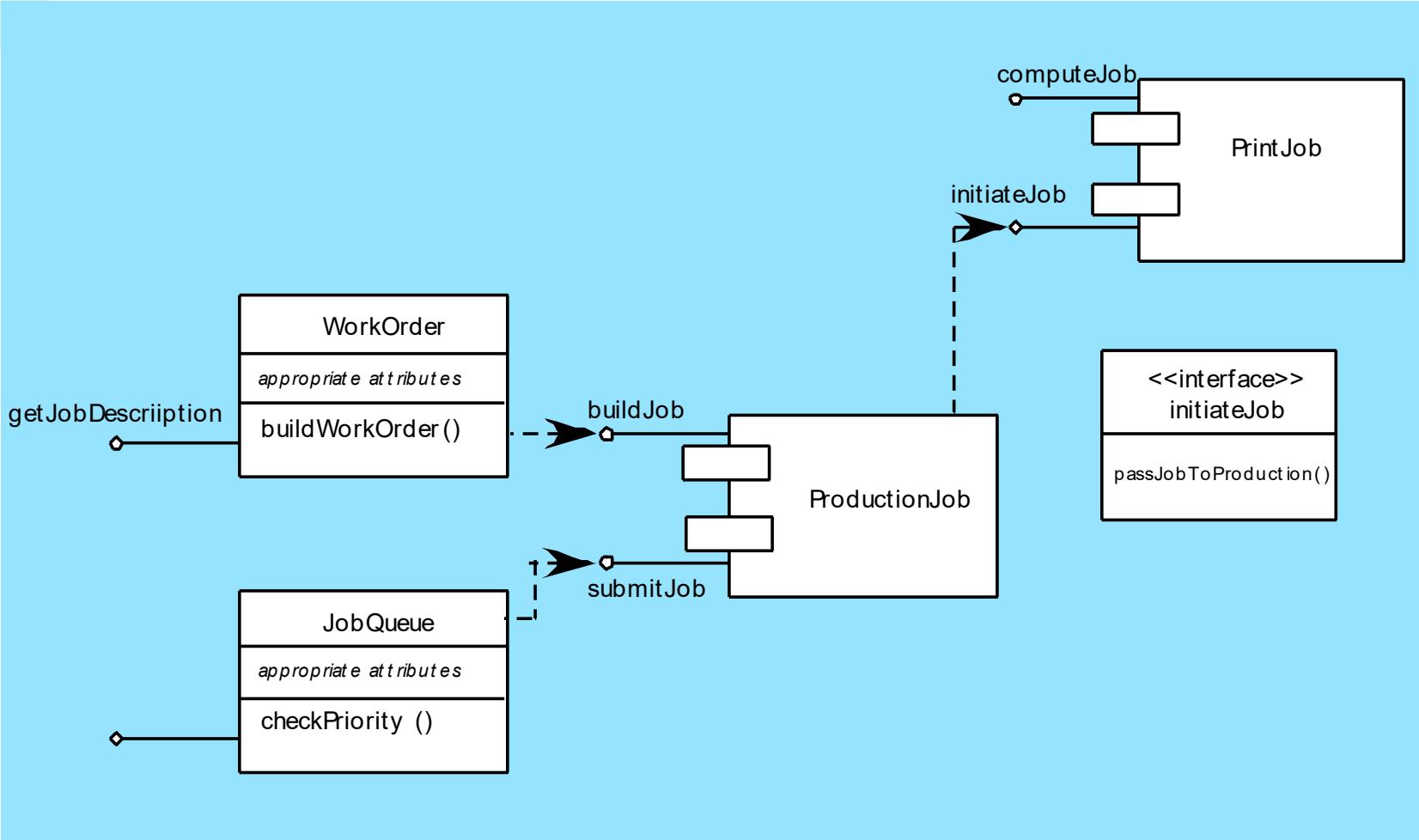
Component Level Design

- Step 1. **Identify** all ***design classes*** that correspond to the problem domain, infrastructure domain.
- Step 2. **Elaborate** all ***design classes*** (not acquired as reusable components).
- Step 2a. Specify ***message*** details when classes or component collaborate.
- Step 2b. Identify appropriate ***interfaces*** for each component.
- Step 2c. Elaborate ***attributes*** and define data types and data structures.
- Step 2d. Describe processing flow within each ***operation*** in detail.
- Step 3. Describe ***persistent*** data sources (databases and files) and identify the ***classes*** required to manage them.
- Step 4. Develop and elaborate ***behavioral*** representations for a class or component.
- Step 5. Elaborate ***deployment*** diagrams to provide additional implementation detail.
- Step 6. **Factor** every component-level design representation and always consider **alternatives**.

Step 2.a. Specify message details when classes or component collaborate – Collaboration Diagram

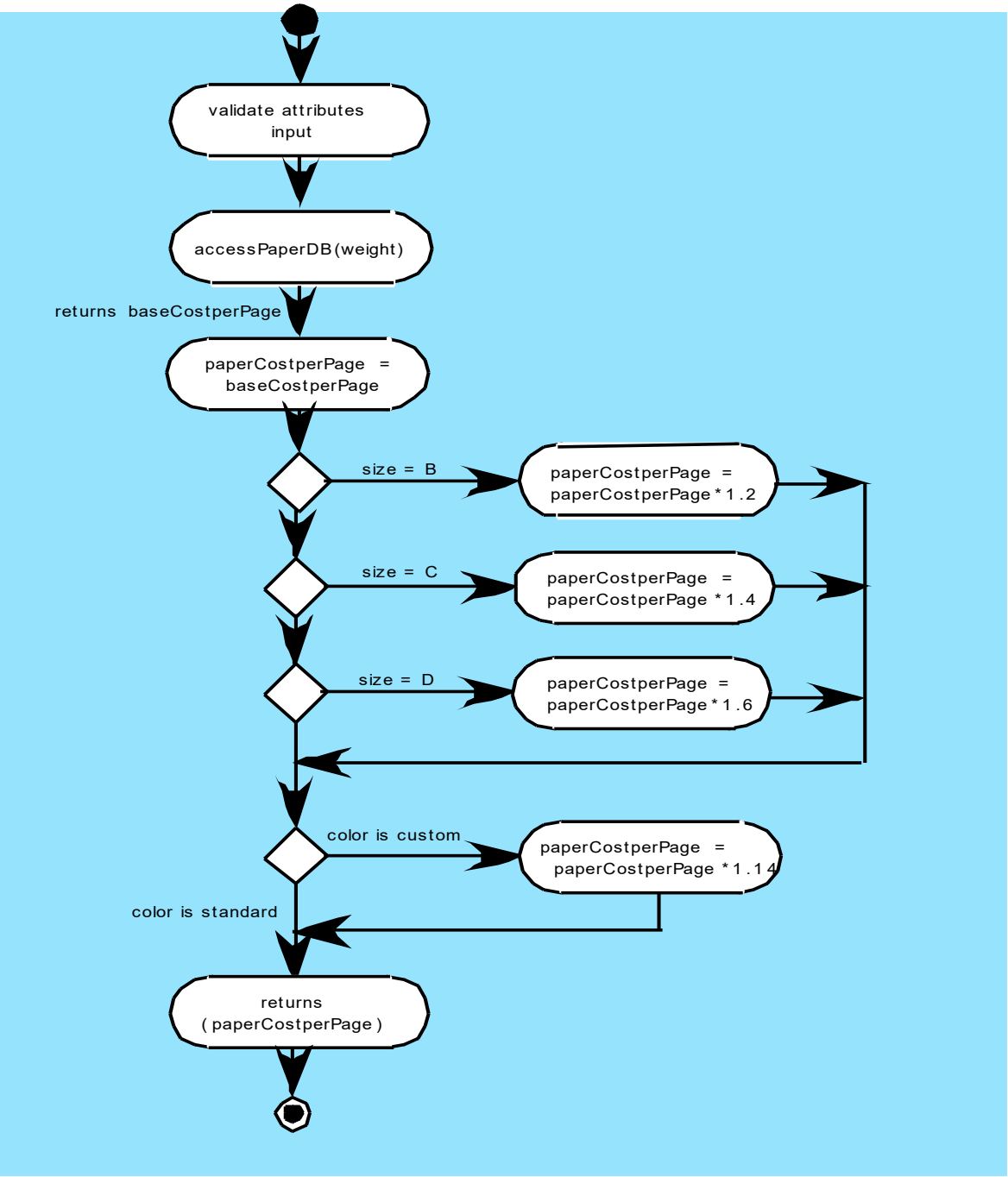


Step 2.b. Identify appropriate interfaces for each component



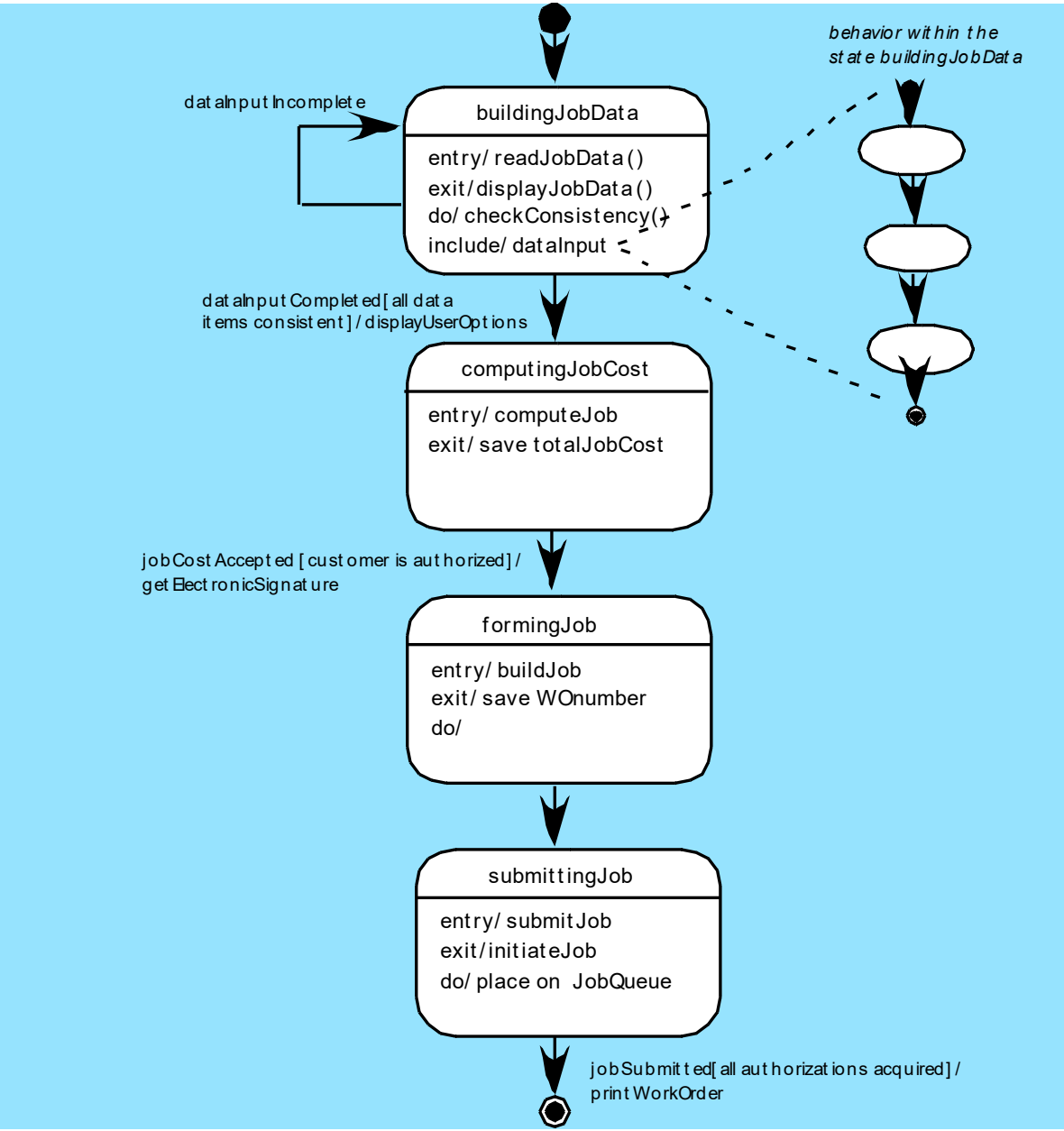
KNOWLEDGE & SOFTWARE ENGINEERING

Step 2.d. Describe processing flow within each operation in detail – Activity Diagram



KNOWLEDGE & SOFTWARE ENGINEERING

Step 4. Develop and elaborate behavioral representations for a class or component – Statechart



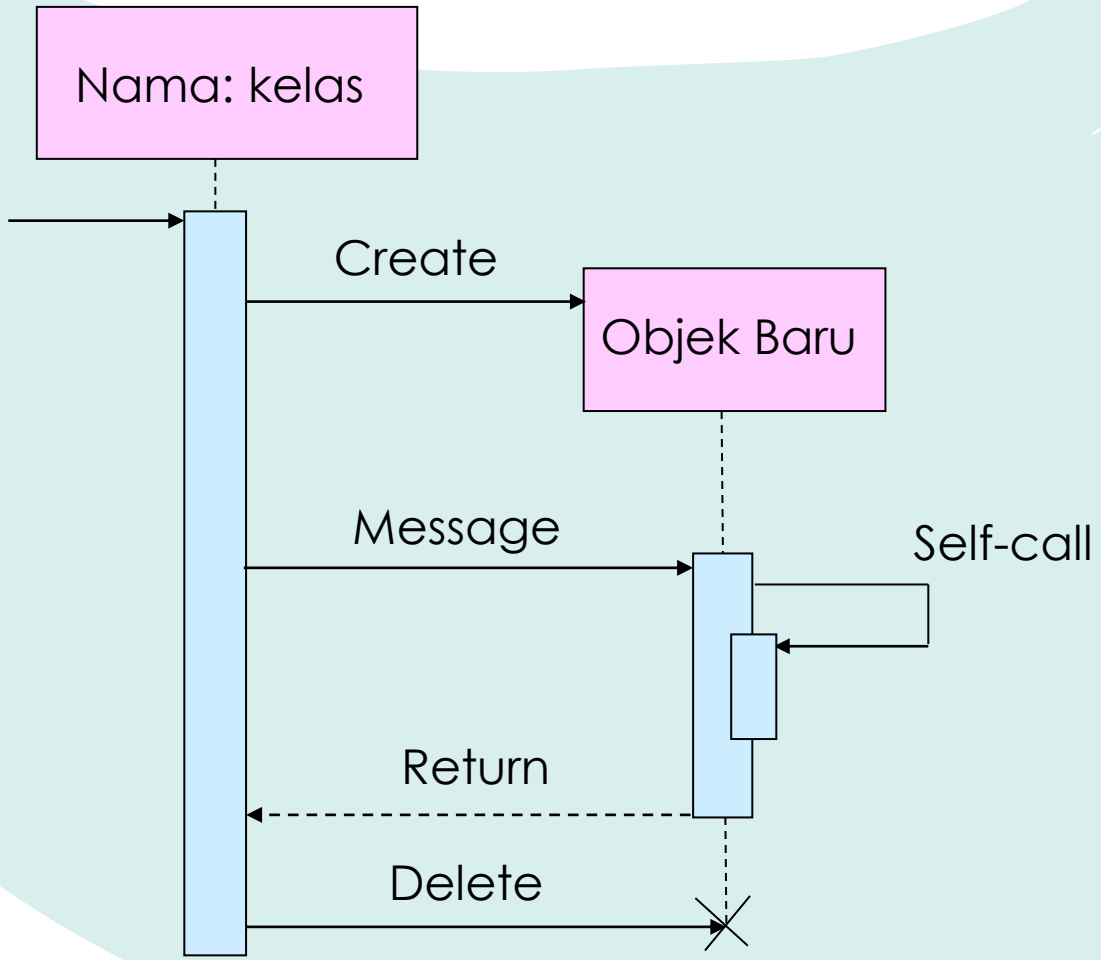
KNOWLEDGE & SOFTWARE ENGINEERING

Step 2.a. Specify message details when classes or component collaborate – Sequence Diagram

Elemen Sequence Diagram

- **Peserta (Participant)/aktor**: suatu objek yang diperlukan pada suatu diagram sekuens
- **Pesan (Message)**: komunikasi antar objek peserta
- Garis vertical arah bawah menunjukkan urutan waktu eksekusi
  - dimulai dari atas, diteruskan ke bawahnya

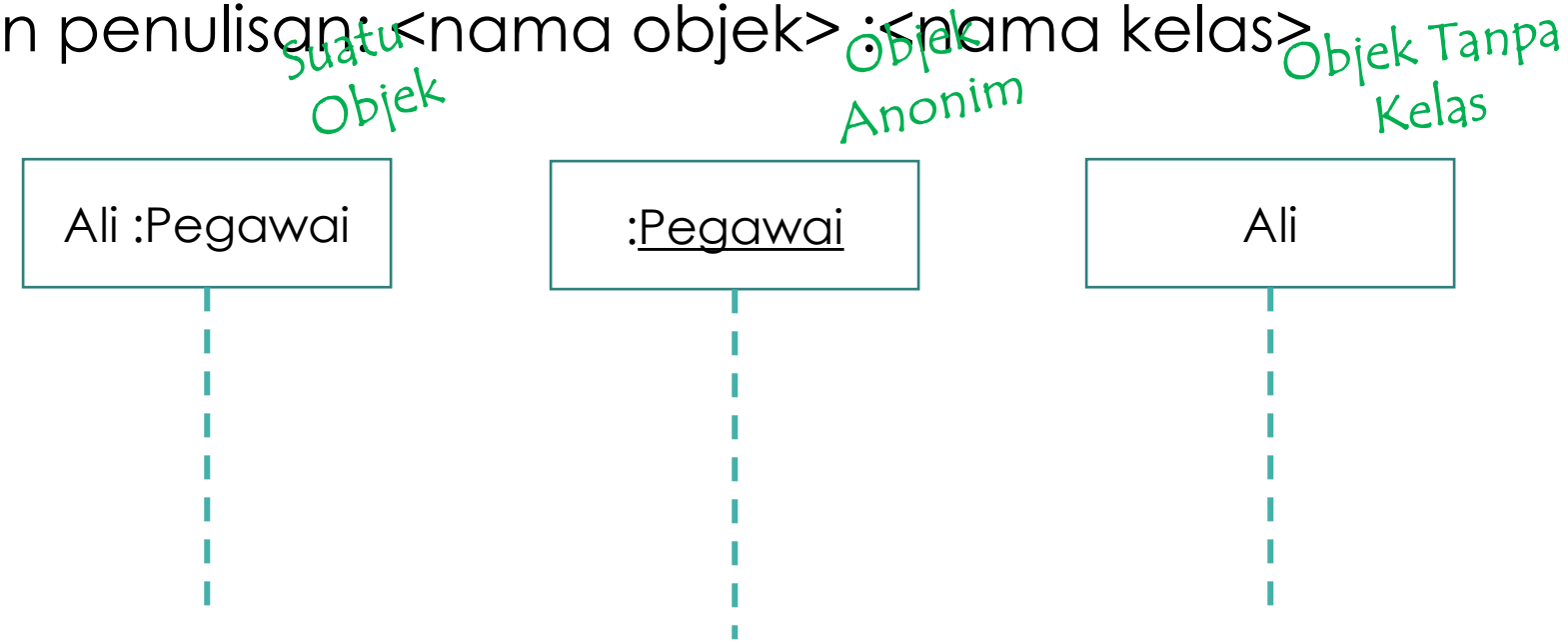
Elemen pada Sequence Diagram



Gambar tsb adalah representasi pemanggilan suatu operasi dengan pengiriman pesan

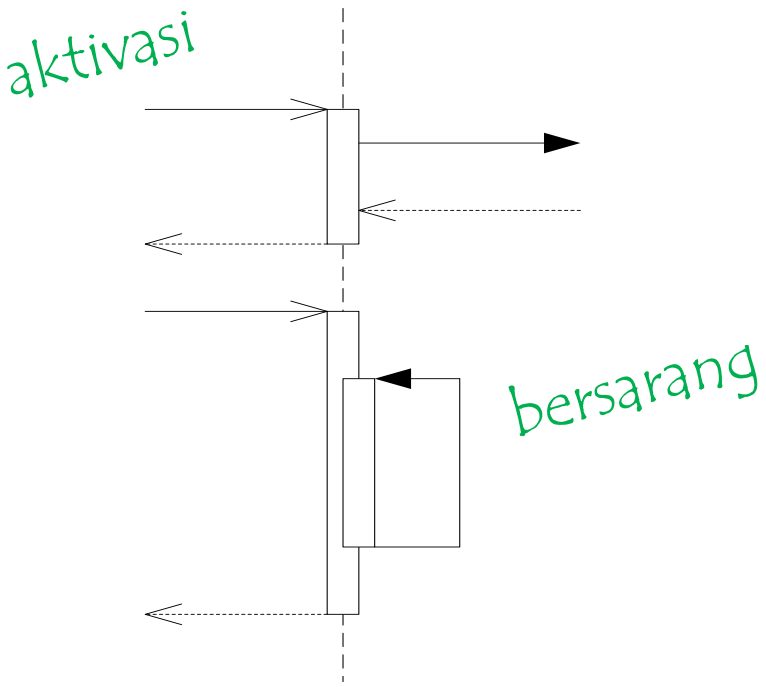
Penamaan Objek

- Penamaan objek dimulai dengan titik dua, lalu nama objek
  - Jika perlu, berikan nama objek untuk memperjelas
  - Garis dibuat terputus-putus
  - Aturan penulisan: <nama objek> :<nama kelas>

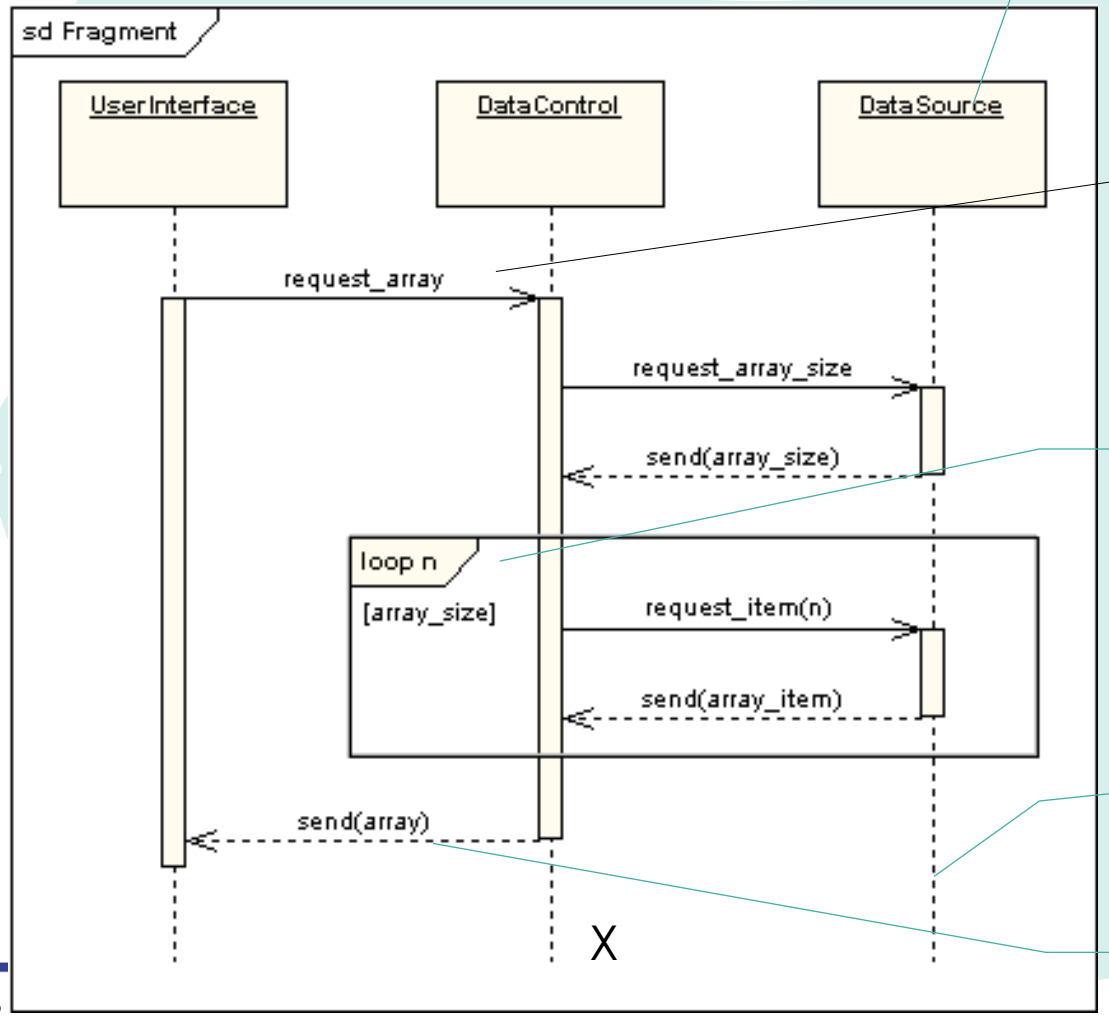


Pemanggilan suatu Metode/Operasi

- **Aktivasi**:
  - Digambarkan sebagai kotak pada garis vertikal
  - Rekursif dapat digambarkan sebagai pemanggilan bersarang



Sequence Diagram



Participants, Object, Class, Actor

Synchronous/Call

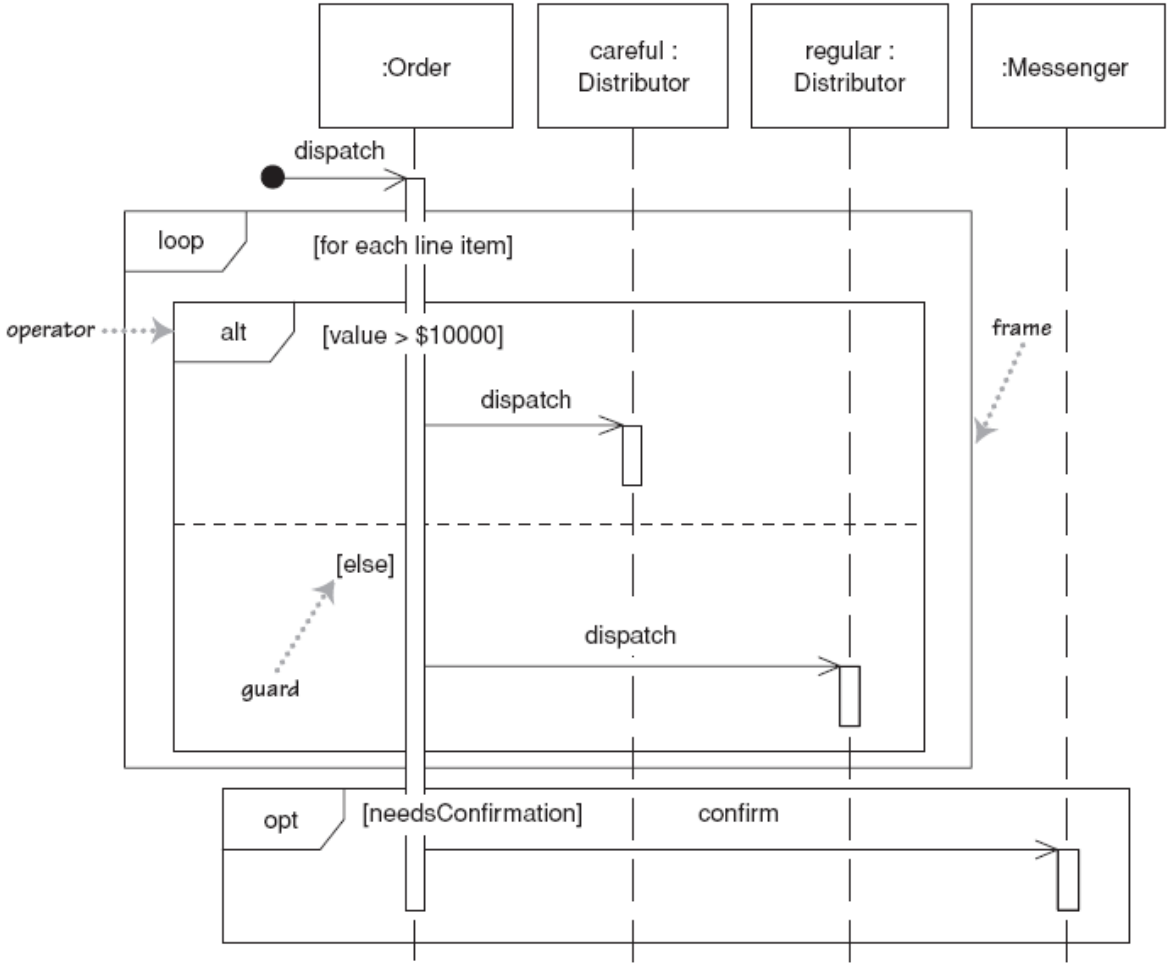
Fragment

Life Line

return

Seleksi, Pengulangan

- Diberikan bingkai: kotak pada diagram sekuens yang mengindikasikan ada seleksi (if) atau pengulangan (loop)
  - `if` -> (opt) [condition]
  - `if/else` -> (alt) [condition], dipisahkan garis putus-putus horisontal
  - `loop` -> (loop) [kondisi atau item yang diulang]

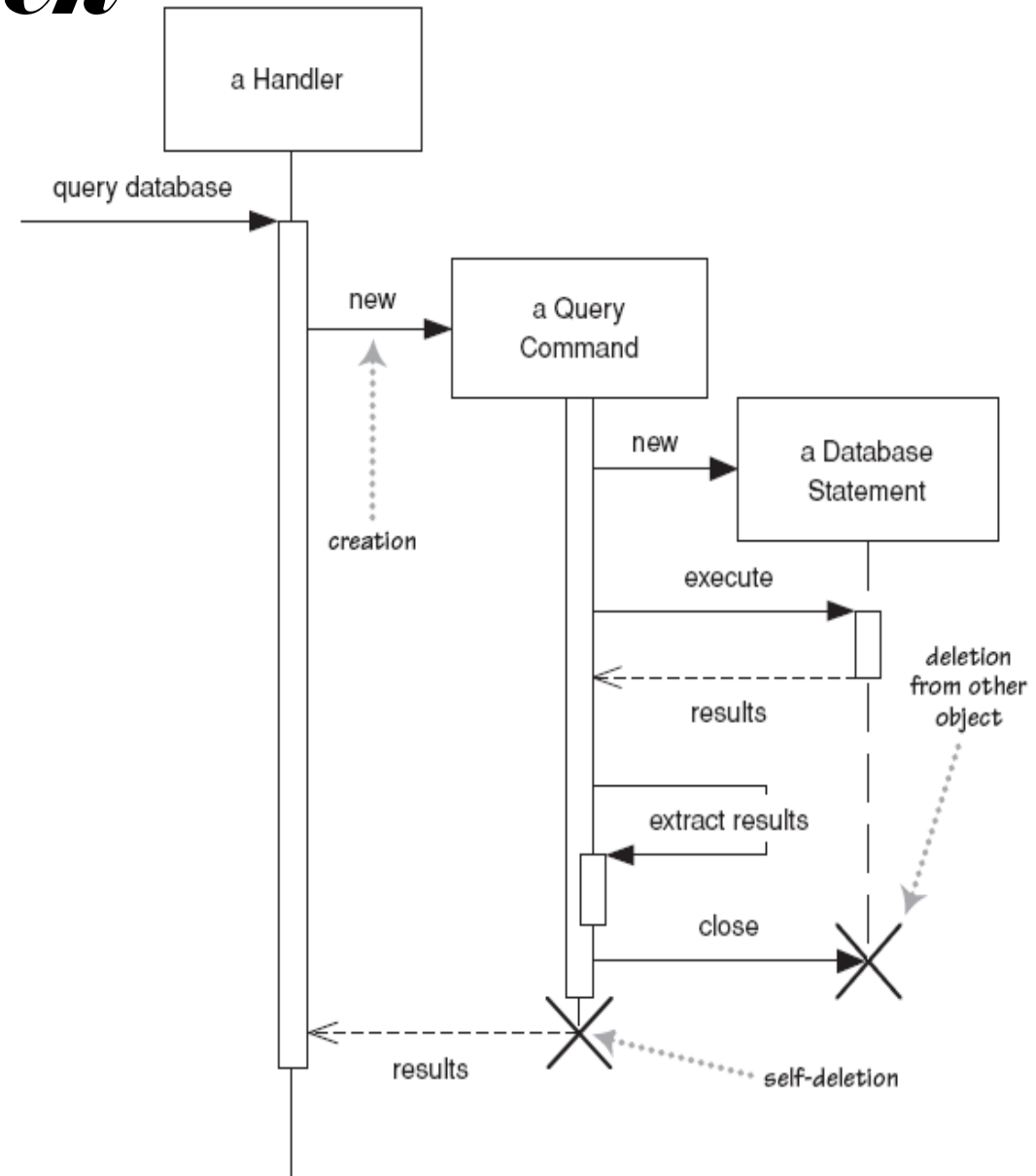


KNOWLEDGE & SOFTWARE ENGINEERING

Daur hidup suatu Objek

- Konstruksi objek *(creation)*
  - Panah dengan tulisan 'new'
  - Perhatikan objek yang dibuat setelah awal suatu scenario akan dituliskan lebih rendah

- Destruksi objek *(deletion)*:
  - X besar di bawah garis horizontal menandai penghapusan objek
  - Pada Bahasa Java, objek tidak secara eksplisit dihapus, setelah keluar dari scope, maka akan ditangani oleh system '*garbage collection*'



KNOWLEDGE & SOFTWARE ENGINEERING

Sequence Diagram dari use case

Basic Course

The Customer specifies an author on the Search Page and then presses the Search button.

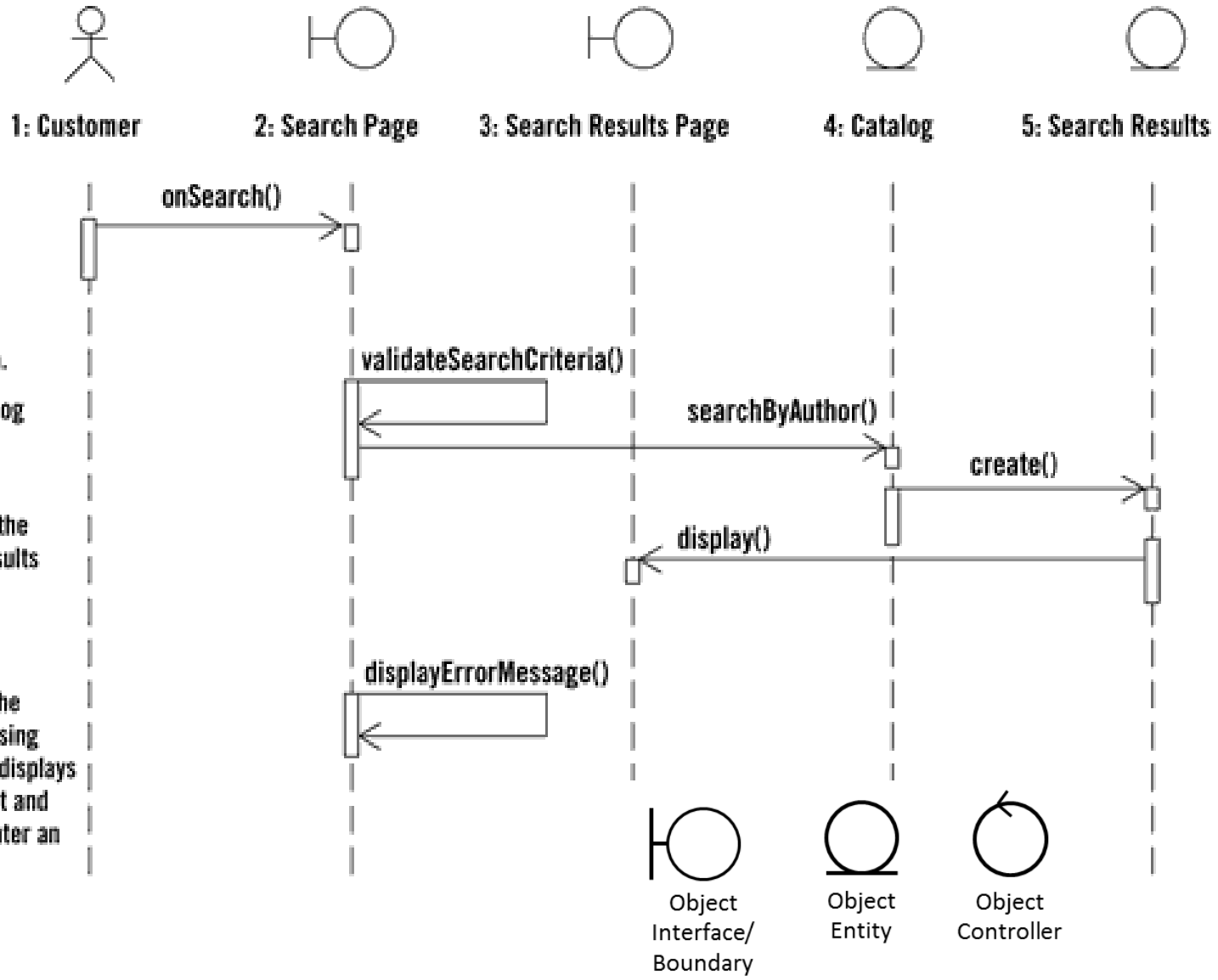
The system validates the Customer's search criteria.

The system searches the Catalog for books associated with the specified author.

When the search is complete, the system displays the search results on the Search Results Page.

Alternate Course

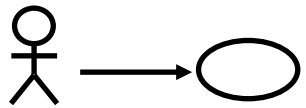
If the Customer did not enter the name of an author before pressing the Search button, the system displays an error message to that effect and prompts the Customer to re-enter an author name.



KNOWLEDGE & SOFTWARE ENGINEERING

Sequence Diagram

- Sequence diagram menampilkan **interaksi objek** yang disusun dalam **urutan waktu**



Registrar Create Course

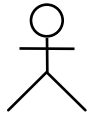
This use case begins after the Registrar logs onto the Registration System with a valid password.

The registrar fills in the course form with the appropriate semester and course related info.

The Registrar requests the system to process the course form.

The system creates a new course, and this use case ends

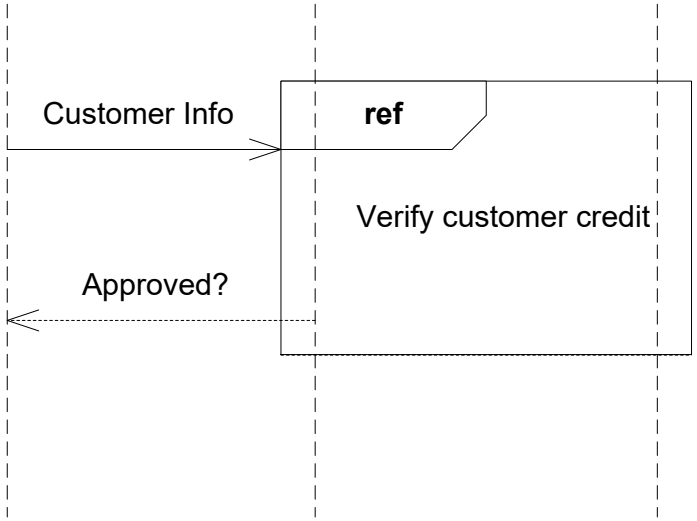
Generic vs Instance

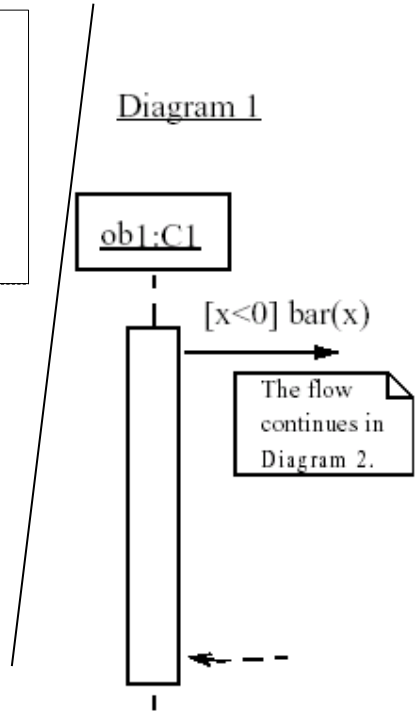


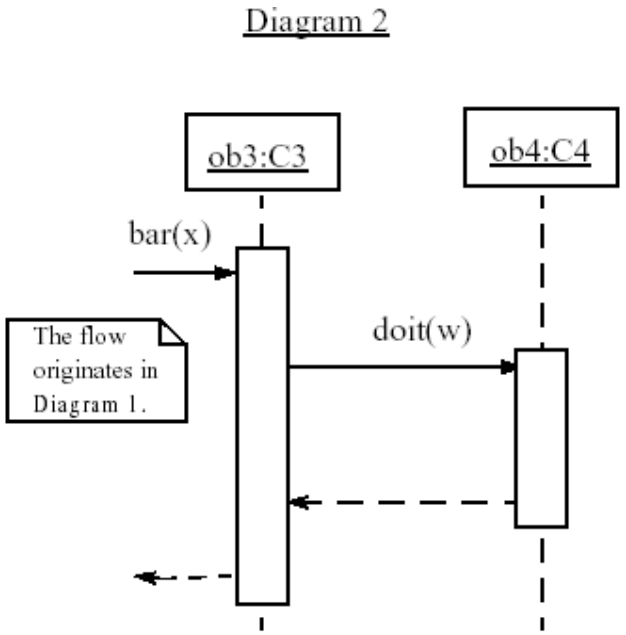
Traceability!

Menghubungkan Sequence Diagram

- Jika sekuens diagram terlalu besar, atau ingin mengacu ke sekuens diagram lagi, maka gunakan:
  - Panah dan juga komentar yang mengacu ke diagram lain
  - Buat bingkai "**ref**" yang diikuti dengan isi nama diagram lain

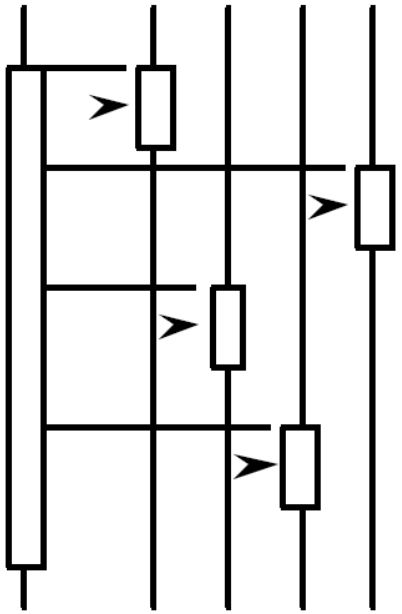


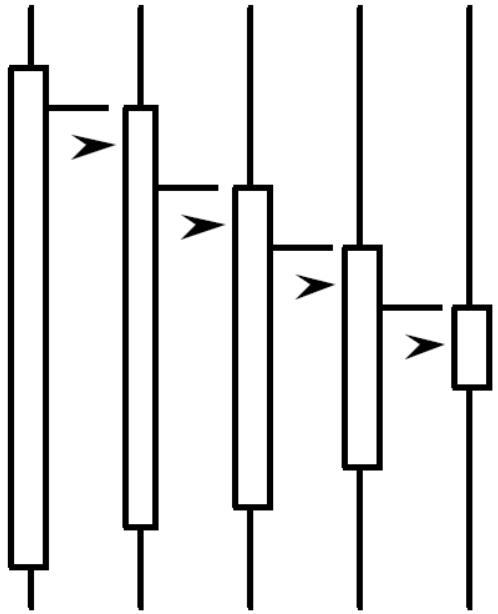




Bentuk-bentuk Sequence Diagram

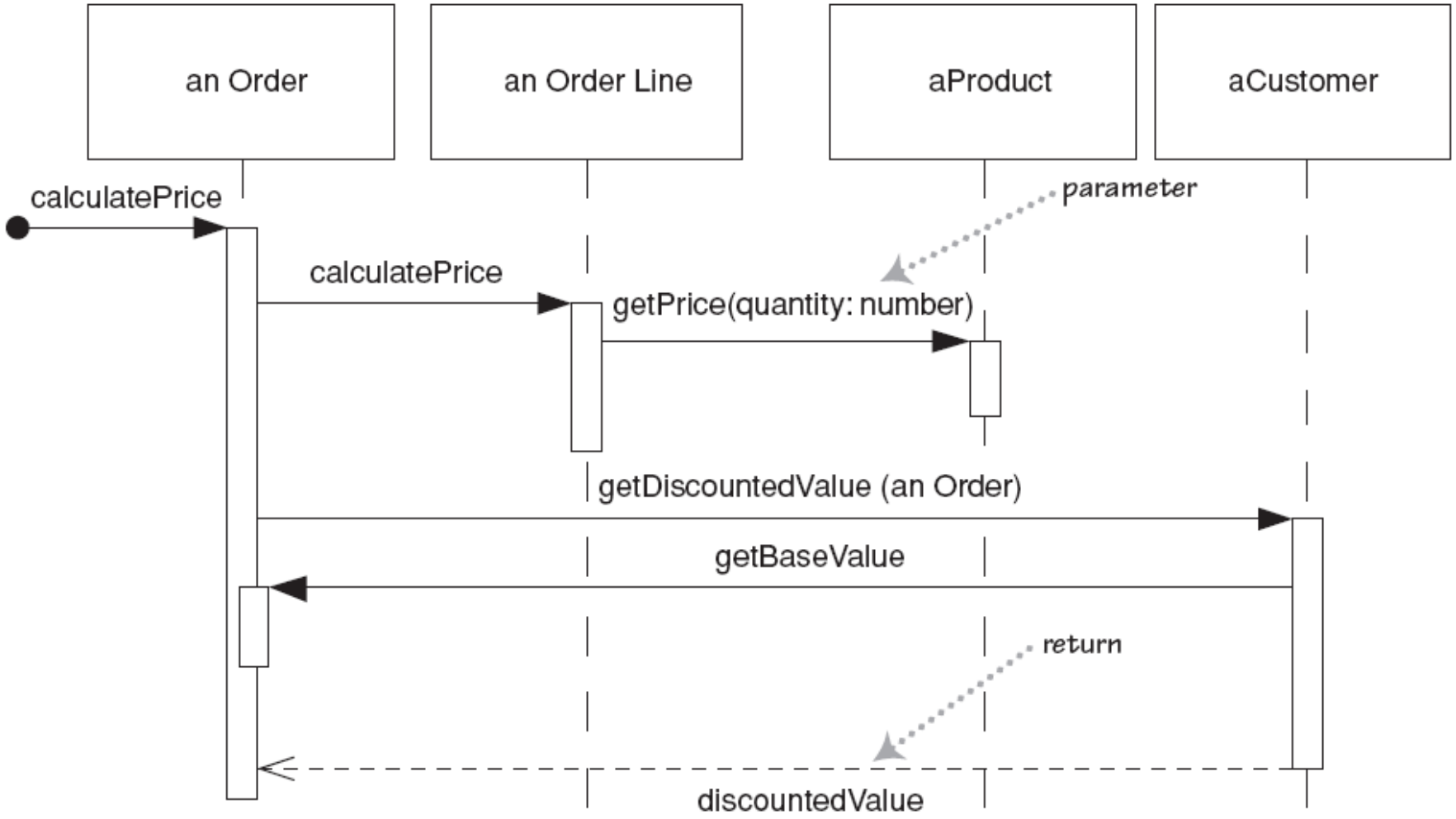
- Lihat aliran panah pada gambar berikut!
  - Terpusat?
  - Tersebar?





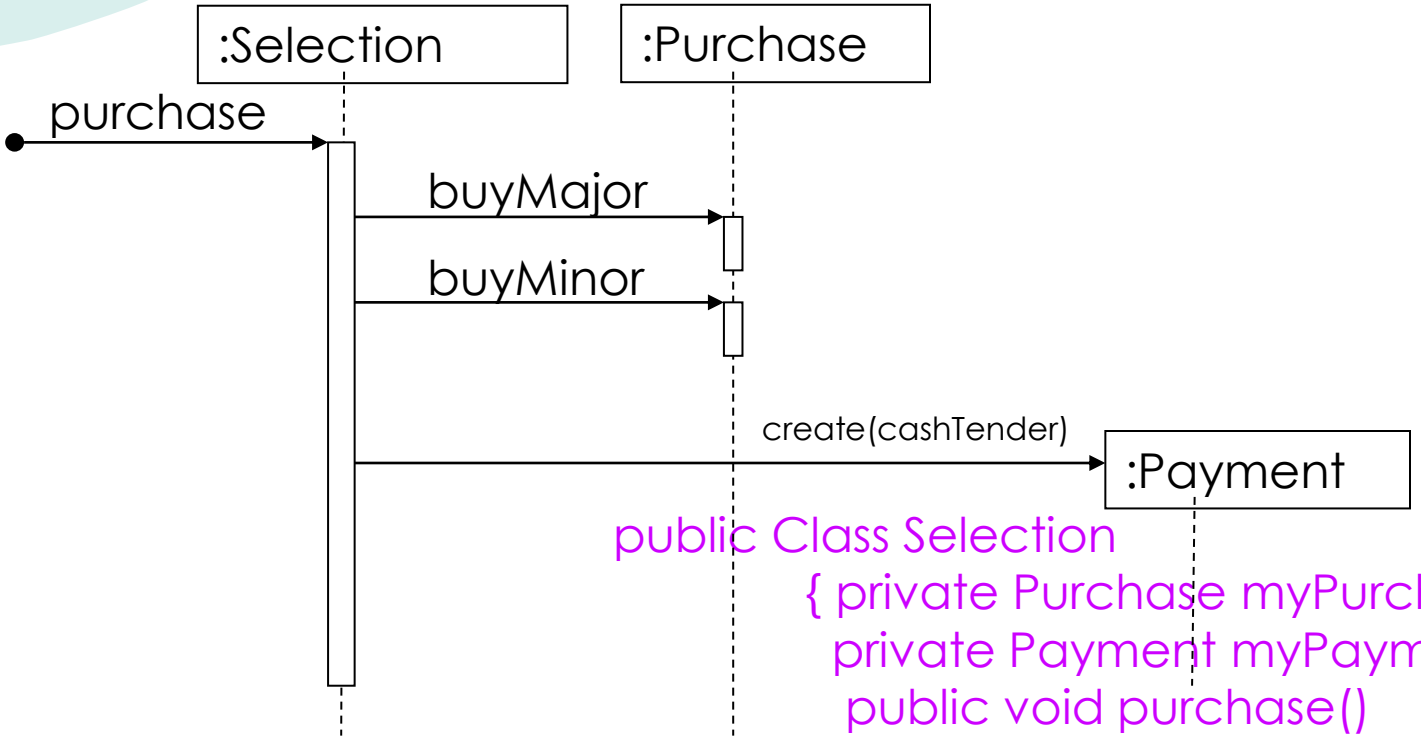
- Pada sistem terpusat (centralized), 1 peserta melakukan seluruh pemrosan
- Pada sistem tersebar (distributed), pemrosesan dibagi ke peserta lain

Contoh Sequence Diagram



KNOWLEDGE & SOFTWARE ENGINEERING

Dari Sequence Diagram ke Pemrograman




```
public Class Selection
    { private Purchase myPurchase = new Purchase();
      private Payment myPayment;
       public void purchase()
            { myPurchase.buyMajor();
              myPurchase.buyMinor():
              myPayment = new Payment( cashTender );
              //. .
            }
      // . .
    }
```

KNOWLEDGE & SOFTWARE ENGINEERING

Kenapa tidak dibuat algoritma saja?

- Sekuens Diagram lebih mudah dimengerti untuk pengguna
  - Tidak semua bagian kode program ditampilkan, sehingga mudah dimengerti
  - Mudah digunakan untuk diskusi dalam 1 tim ataupun dengan pihak lain
- Keterlibatan Objek/Kelas dapat terlihat pada halaman yang sama
- Tidak tergantung Bahasa

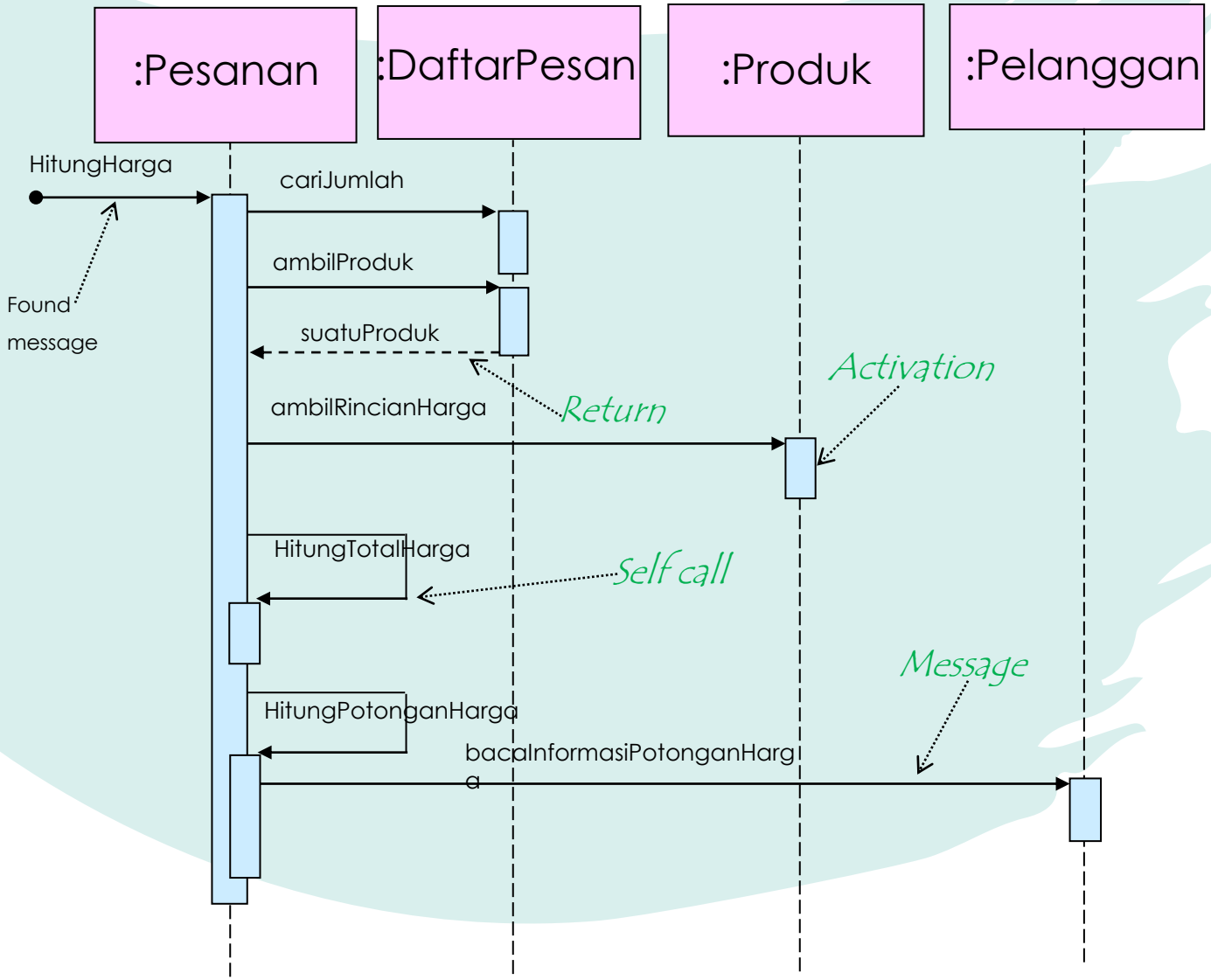
Contoh Diagram Sekuens [Fowler]

Ada pesanan datang, sistem akan memanggil suatu perintah untuk menghitung harganya. Pesanan ini memiliki daftar pesan. Lalu untuk menentukan harganya, akan diambil dari data produk. Kemudian dari pesanan tersebut, akan dihitung total harganya, sesudahnya potongan harga akan diberikan berdasarkan kategori pelanggan

Contoh Diagram Sekuens [Fowler]

Ada **pesanan** datang, sistem akan memanggil suatu perintah untuk menghitung harganya. Pesanan ini memiliki **daftar pesan**. Lalu untuk menentukan harganya, akan diambil dari data **produk**. Kemudian dari pesanan tersebut, akan dihitung total harganya, sesudahnya potongan harga akan diberikan berdasarkan kategori **pelanggan**

Contoh Diagram Sekuens



KNOWLEDGE & SOFTWARE ENGINEERING

Designing traditional component

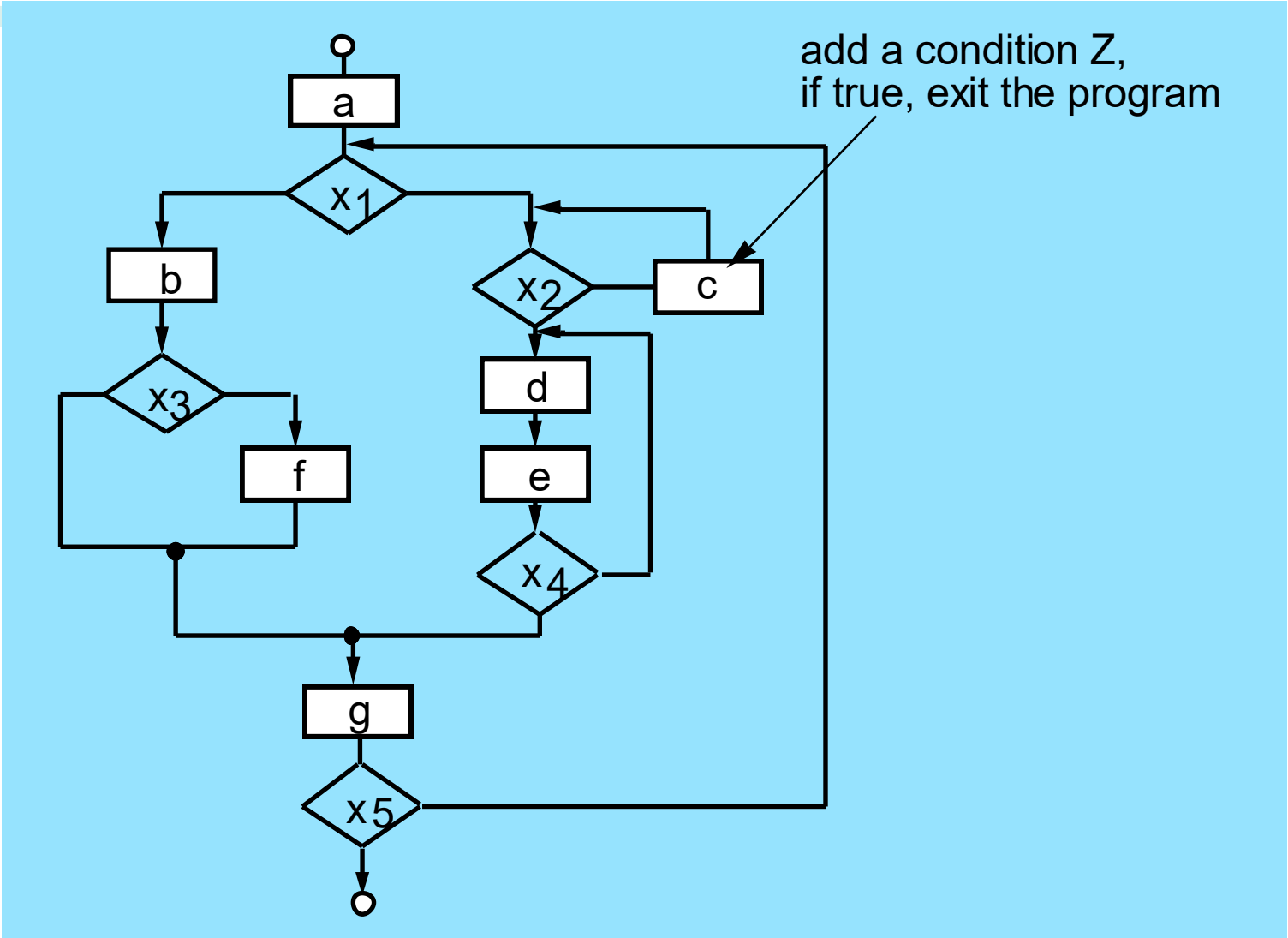
Algorithm Design

- the closest design activity to coding
- the approach:
  - review the design description for the component
  - use *stepwise refinement* to develop algorithm
  - use *structured programming* to implement procedural logic
  - use 'formal methods' to prove logic

Algorithm Design Model

- represents the algorithm at a level of detail that can be reviewed for quality
- options:
  - graphical (e.g. flowchart, box diagram)
  - pseudocode (e.g., PDL) … choice of many
  - programming language(?)
  - decision table
  - conduct walkthrough to assess quality

A Structured Procedural Design



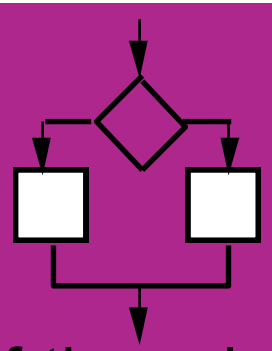
KNOWLEDGE & SOFTWARE ENGINEERING

Decision Table

	Rules					
Conditions	**1**	**2**	**3**	**4**	**5**	**6**
regular customer	T	T				
silver customer			T	T		
gold customer					T	T
special discount	F	T	F	T	F	T
Rules						
no discount	✓					
apply 8 percent discount			✓	✓		
apply 15 percent discount					✓	✓
apply additional x percent discount		✓		✓		✓

KNOWLEDGE & SOFTWARE ENGINEERING

Program Design Language (PDL)



if-then-else

```
if condition x
  then process a;
  else process b;
endif
```

PDL

- easy to combine with source code
- machine readable, no need for graphics input
- graphics can be generated from PDL
- enables declaration of data as well as procedure
- easier to maintain

KNOWLEDGE & SOFTWARE ENGINEERING